

```
// Service worker minimale: memorizza in
cache solo l'involucro dell'app
// (pagina, manifest, icone) per un avvio
più rapido e un minimo di
// funzionamento offline. I dati veri e
propri (Firestore, Firebase,
// caratteri e librerie da CDN) NON vengono
mai serviti dalla cache: per
// quelli si va sempre in rete, così non si
rischia mai di vedere dati
// vecchi o mancanti.
const NOME_CACHE = "diario-batteria-v1";
const APP_SHELL = [
  "./",
  "./index.html",
  "./manifest.json",
  "./icon-192.png",
  "./icon-512.png"
];

self.addEventListener("install", (evento)
=> {
  evento.waitUntil(
    caches.open(NOME_CACHE)
      .then((cache) =>
cache.addAll(APP_SHELL))
      .catch(() => {})
  );
  self.skipWaiting();
});
```

```

self.addEventListener("activate", (evento)
=> {
    evento.waitUntil(
        caches.keys().then((nomi) =>
            Promise.all(nomi.filter((n) => n !==
NOME_CACHE).map((n) => caches.delete(n)))
        )
    );
    self.clients.claim();
});

self.addEventListener("fetch", (evento) =>
{
    const richiesta = evento.request;
    if (richiesta.method !== "GET") return;

    const url = new URL(richiesta.url);
    // Lascia passare tutto ciò che non è
l'app stessa (Firestore, Firebase,
    // Google Fonts, CDN dei
caratteri/librerie): deve sempre arrivare
    // fresco dalla rete, mai dalla cache.
    if (url.origin !== self.location.origin)
return;

    evento.respondWith(
        caches.match(richiesta).then((risposta)
=> {
            if (risposta) return risposta;
            return fetch(richiesta)
                .then((rete) => {
                    const copia = rete.clone();

```

```
    caches.open(NOME_CACHE).then((cache) =>
      cache.put(richiesta, copia));
      return rete;
    })
    .catch(() => risposta);
  })
);
});
```